

Introducción. Primeros pasos.

1 Introducción 2

2 Crear una infraestructura de MongoDB en la nube con MongoDB Atlas. 2

3 Conexión al clúster desde el cliente Compass. 5

4 Primeros pasos 8

4.1 Sentencias básicas 8

4.2 Sentencias CRUD en MongoDB 10

4.2.1 Búsqueda de documentos 10

4.2.1.1 El método .find() 10

4.2.2 Inserción de documentos 13

4.2.2.1 El campo _id 13

4.2.2.2 El método .insertOne() 13

4.2.2.3 El método .insertMany() 13

4.2.3 Modificación de documentos 13

4.2.3.1 El método .updateOne() 13

4.2.3.2 El método .updateMany() 15

4.2.3.3 El método .replaceOne() 15

4.3 Borrado de documentos 15

4.3.1.1 El método .deleteOne() 15

4.3.1.2 El método .deleteMany() 15

5 Operaciones de agregación. Método .aggregate() 16

5.1 Cómo realizar operaciones de Join en MongoDB 17

1 Introducción

MongoDB es un sistema de gestión de bases de datos NoSQL orientado a documentos. A diferencia de las bases de datos relacionales tradicionales (como MySQL o PostgreSQL), MongoDB almacena los datos en documentos **JSON** o **BSON** (Binary JSON), lo que le otorga flexibilidad y escalabilidad.

Se utiliza ampliamente en aplicaciones modernas debido a su capacidad para manejar grandes volúmenes de datos, su escalabilidad horizontal y su facilidad de integración con tecnologías web y en la nube.

Administrar un sistema **MongoDB** implica tareas similares a las de otros sistemas de gestión de bases de datos (SGBD), aunque con particularidades debido a su naturaleza **NoSQL**:

- Instalación y configuración.
- Gestión de usuarios y permisos.
- Gestión y organización de bases de datos y colecciones.
- Gestión de índices.
- Copias de seguridad y recuperación.
- Configuración de la replicación y alta disponibilidad.
- Monitorización y optimización de rendimiento.

2 Crear una infraestructura de MongoDB en la nube con MongoDB Atlas.

Dado que estamos en el módulo de ***Lenguajes de Marcas*** y no de ***Gestión de Bases de Datos***, vamos a ver cómo interactuar con sistemas MongoDB, tratando de dedicar el menor tiempo posible a tareas de índole administrativo. Por ello, vamos a optar por crear una infraestructura en la nube a través de la plataforma ***MongoDB Atlas***.

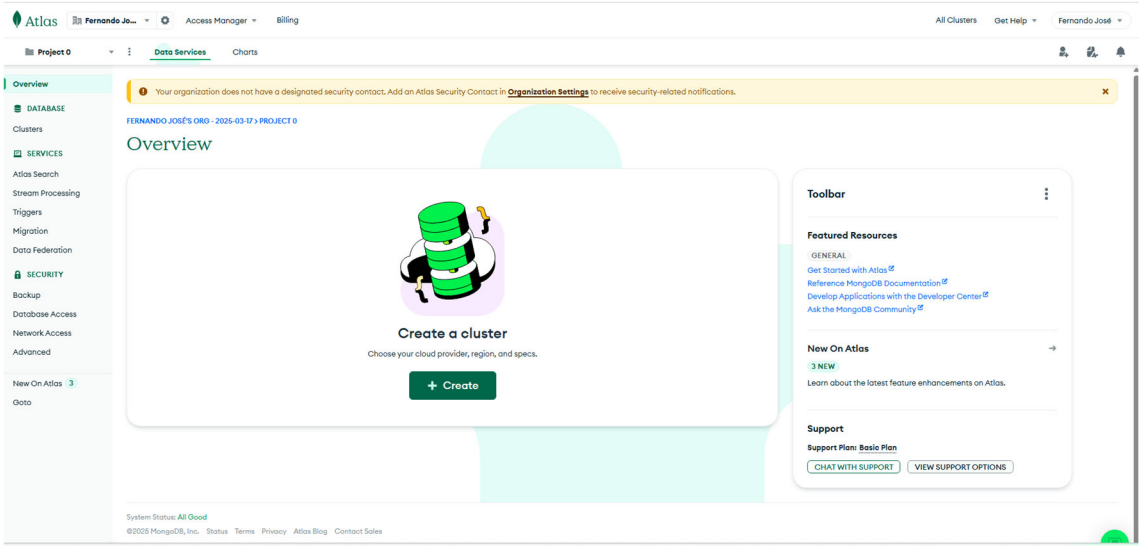
MongoDB Atlas es un servicio de **base de datos como servicio** completamente administrado que permite ejecutar **MongoDB en la nube** sin preocuparnos por la infraestructura. Es ofrecido por *MongoDB Inc*. y está disponible en **Amazon Web Services (AWS), Google Cloud Platform (GCP) y Microsoft Azure**.

https://www.mongodb.com/es/atlas

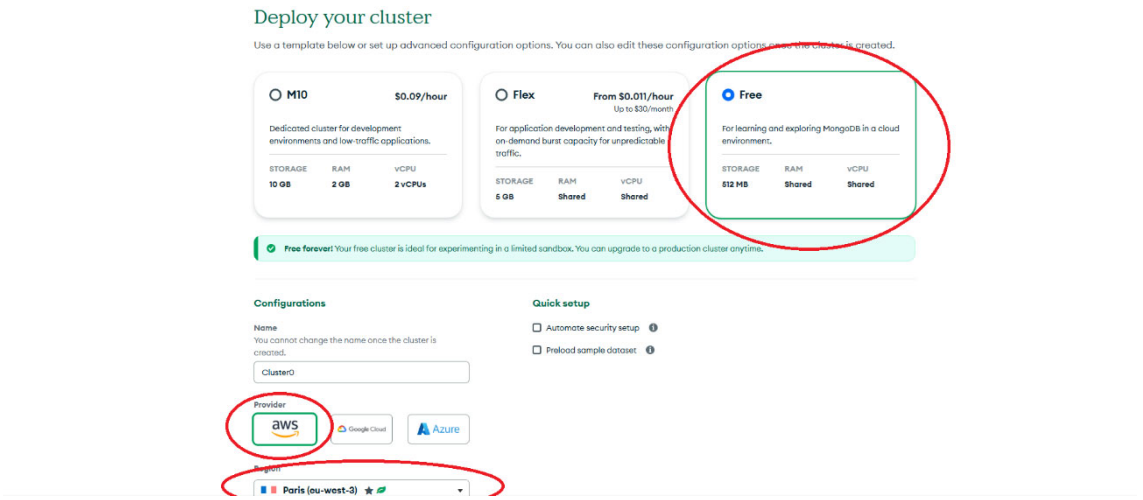


Lo primero que tenemos que hacer es darnos de alta en la plataforma y acceder con dicho usuario. MongoDB Atlas ofrece una gran versatilidad a la hora de construir nuestra infraestructura. La mayoría de las opciones son de pago, aunque también nos ofrece la posibilidad de lanzar una infraestructura sencilla de forma gratuita.

Una vez que hemos hecho *login* con nuestro usuario, veremos algo parecido a lo siguiente:



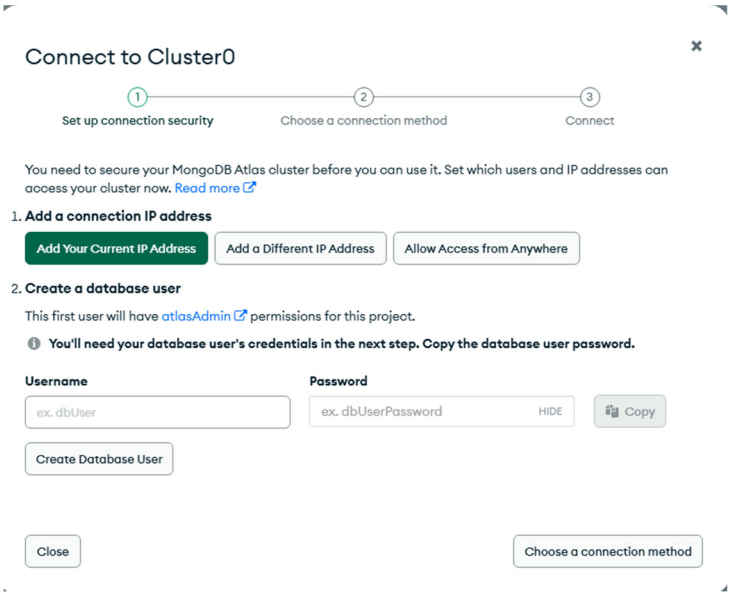
Lo que haremos a continuación **es crear un nuevo clúster**. De las opciones que se nos ofrecen, elegiremos la gratuita:



El proveedor de la nube y la región dónde se creará el clúster podemos dejarlo con los valores por defecto. Debemos ponerle el nombre que queramos (por defecto ***Cluster0***).

A continuación, se nos preguntarán ciertos valores de configuración:

1 – **Cómo nos conectaremos al clúster**



En este punto tenemos que especificar si queremos que a nuestro clúster se pueda acceder desde cualquier dirección IP o restringir el acceso desde una IP determinada. Por seguridad, elegiremos la opción de ***"Add Your Current IP Address"***.

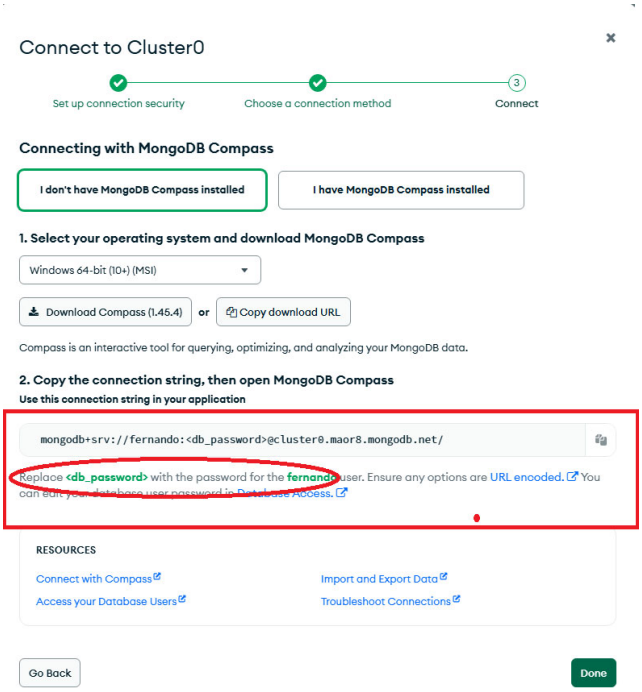
También tenemos que especificar si las credenciales para del usuario con el que nos conectaremos al clúster. Tras poner su identificador y contraseña pulsaremos sobre el botón ***"Create Database User"***.

2- Le indicamos a través de qué cliente nos conectaremos al clúster. En esta ventana nos indica cómo podemos conectarnos al clúster desde diferentes tipos de clientes:

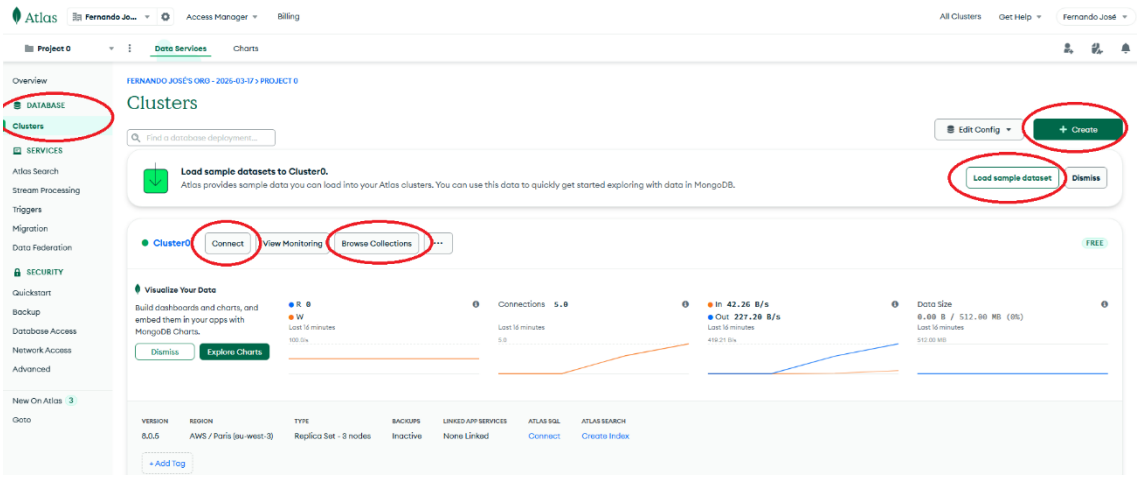
- **Drivers**. Si queremos interactuar desde código escrito en diferentes lenguajes de programación.
- **Compass**. Es el cliente gráfico oficial. Es la opción que elegiremos en nuestro caso.
- **Shell**. Si nos conectamos desde un cliente de consola.
- **MongoDB for VS Code**. Si queremos conectarnos desde este entorno de programación.
- **Atlas SQL**.

En la ventana que aparece al pulsar sobre la opción ***Compass***, se nos da la posibilidad de descargarnos la aplicación si todavía no la tenemos. También, nos indica el string de conexión que hemos de usar para conectarnos al clúster:

OJO, no debemos olvidar sustituir <db_password> por la contraseña establecida para el usuario.



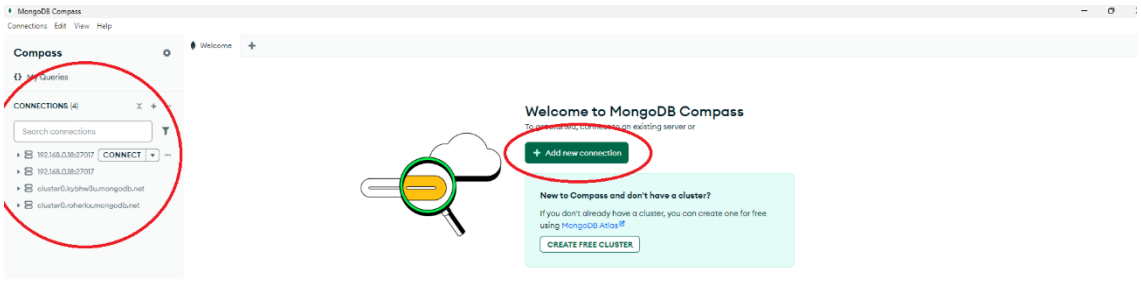
Una vez que tenemos el clúster creado, podemos ir a la opción **"Clusters"** dentro de **"DATABASE"** para ver el estado de los clústers que tenemos en ese momento. Desde ahí, podremos crear nuevos clústers, ver las opciones de conexión para un clúster determinado o las *colecciones ("equivalentes" a tablas)* que tenemos definidas en ese momento. También podemos cargar un dataset de ejemplo para tener algún dato sobre el que interactuar.



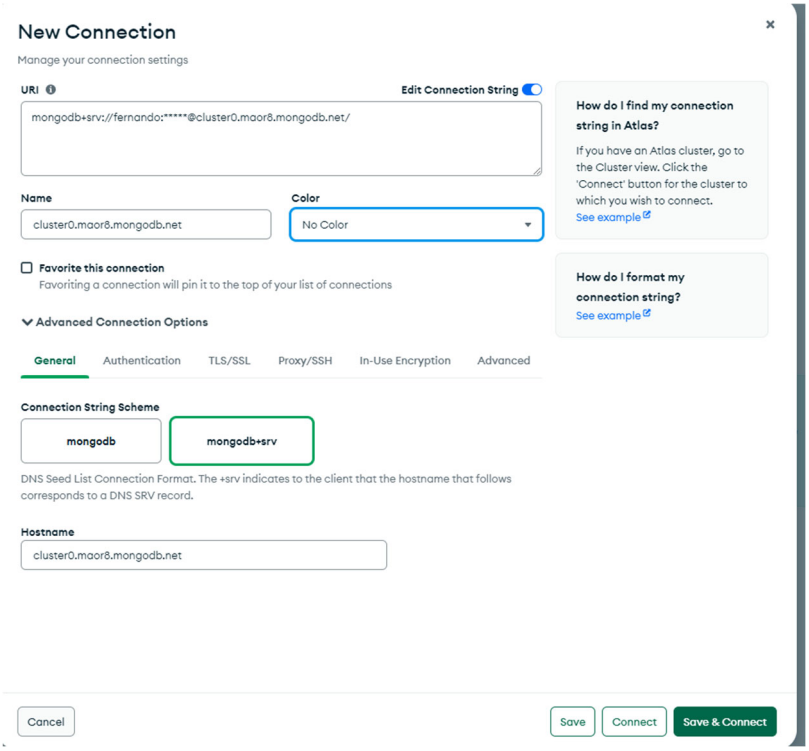
3 Conexión al clúster desde el cliente Compass.

Una vez que tenemos nuestro clúster de MongoDB Atlas en marcha, estamos en disposición de conectarnos al mismo desde nuestro cliente gráfico ***Compass***, que tenemos que tener instalado previamente.

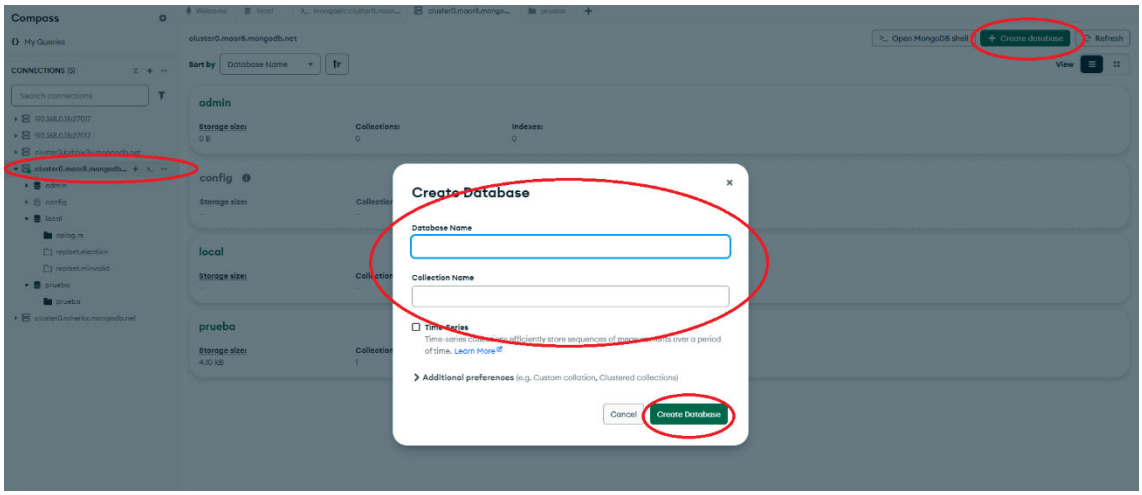
Lanzamos la aplicación y pinchamos sobre la opción ***"Add new connection".*** En el panel de la izquierda, podemos ver las conexiones que ya hemos definido previamente:



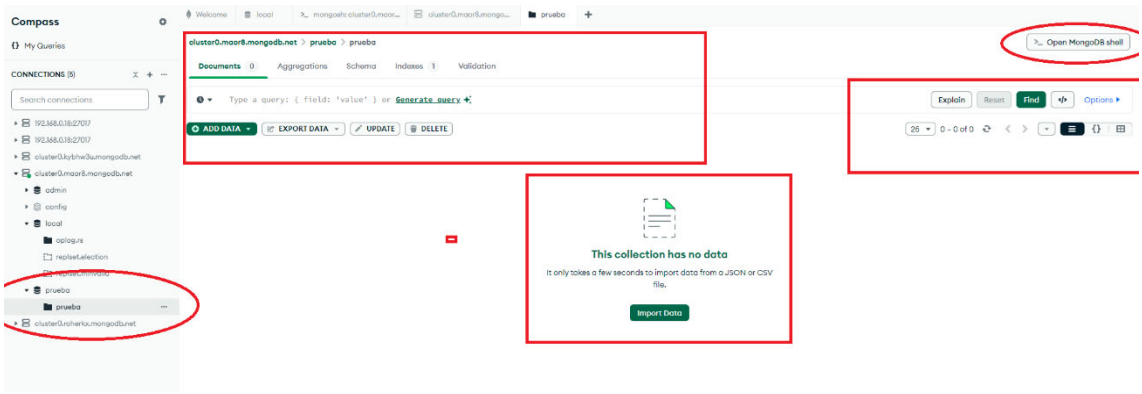
En el cuadro que se muestra, tenemos que poner en el cuadro para la URI, la cadena de conexión copiada anteriormente con el identificador del usuario y la contraseña:



Una vez que nos hemos conectado, aparecerá en el panel de la izquierda la conexión como activa y podremos crear desde ella una ***nueva base de datos*** con una nueva *colección* inicial.



Una vez completado, pinchamos sobre la *colección* que acabamos de crear y ya podremos interactuar con ella. La interacción se puede hacer desde diferentes partes:



- **Opciones de la interface gráfica**.

- **Abriendo un Shell de MongoDB**.



Para poder trabajar y hacer consultas básicas, vamos a cargar datos en la *colección* desde el botón ***Import Data***, donde seleccionaremos el fichero JSON en el que tenemos nuestros datos. Si tenemos un fichero JSON con varios objetos podemos usar uno de estos formatos:

- **Opción 1 (como una sucesión de objetos sin coma como separador)**:

  {objeto1}
  {objeto2}
  ....
  {objetoN}
  OJO, sin usar comas entre los distintos objetos.

- **Opción 2 (como un array de objetos)**:
  [{objeto1},
  {objeto2},
  …
  {objetoN}
  ]
  OJO, en este caso si habría que usar comas entre objetos.

4 Primeros pasos

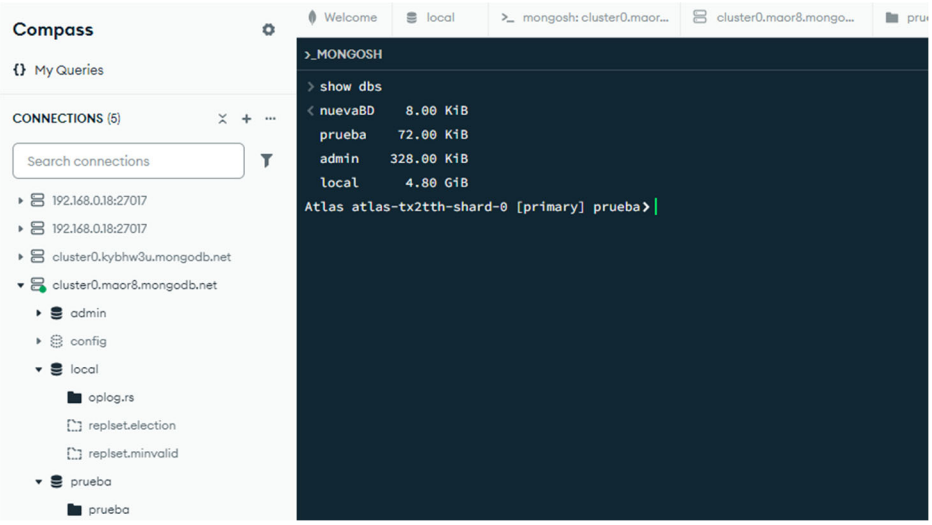
Nota aclaratoria.

No resulta sencillo sintetizar todas las posibilidades y sentencias que permite el lenguaje de consultas de *MongoDB*. Lo que se incluye en los siguientes apartados es un resumen de cómo utilizar las sentencias esenciales, las más frecuentes a la hora de trabajar sobre esta herramienta. Además, no debemos olvidar que estamos en el módulo de LMSGI y no de GBD.

Para una referencia completa, deberíamos buscar en la documentación oficial de *MongoDB*:

https://www.mongodb.com/docs/manual/

Para hacer las pruebas vamos a abrir el Shell, ya que la escritura de sentencias es más ágil desde la terminal.



4.1 Sentencias básicas

Ver BDS

En el *prompt* podemos ver qué *base de datos* está activa en ese momento. Si queremos saber todas las *bases de datos* que hay en el servidor, podemos ejecutar la sentencia:

show dbs

```
>_MONGOSH
> show dbs
< nuevaBD    8.00 KiB
  prueba    72.00 KiB
  admin    328.00 KiB
  local      4.80 GiB
Atlas atlas-tx2tth-shard-0 [primary] prueba>
```

Activar BD

Para poner una *base de datos* como *activa* utilizamos el comando ***use***. Por ejemplo:

use prueba

Aparte de en el prompt, podemos ver la base de datos activa en un momento dado con el comando:

db

Crear BD

Si queremos crear una *base de datos* nueva debemos poner previamente:

use nuevaBD

Sin embargo, la *base de datos* no se creará hasta que creemos algo dentro.

Crear colección

Las bases de datos *MongoDB* se organizan en *colecciones*. Una *colección* sería el equivalente al concepto de *tabla* del *modelo relacional*. Dentro de una *colección*, los *documentos* pueden tener cualquier formato. **Para crear una en la *base de datos activa*** utilizaremos el comando:

db.createCollection(name, [options])

```
db.createCollection( <name>,
   {
     capped: <boolean>,
     timeseries: {                 // Added in MongoDB 5.0
        timeField: <string>,       // required for time series collections
        metaField: <string>,
        granularity: <string>
     },
     expireAfterSeconds: <number>,
     clusteredIndex: <document>,    // Added in MongoDB 5.3
     changeStreamPreAndPostImages: <document>,  // Added in MongoDB 6.0
     autoIndexId: <boolean>,
     size: <number>,
     max: <number>,
     storageEngine: <document>,
     validator: <document>,
     validationLevel: <string>,
     validationAction: <string>,
     indexOptionDefaults: <document>,
     viewOn: <string>,             // Added in MongoDB 3.4
     pipeline: <pipeline>,         // Added in MongoDB 3.4
     collation: <document>,        // Added in MongoDB 3.4
     writeConcern: <document>
   }
)
```

Por ejemplo:

```
db.createCollection("col1")
```

Mostrar colecciones

Para ver qué *colecciones* están definidas en la *base de datos activa* ejecutaría:

```
show collections
```

Borrar BD

Si lo que me interesa es borrar la *base de datos activa*, usaría el comando:

```
db.dropDatabase()
```

Borrar colección

Si lo que quiero es borrar una *colección* de la *base de datos activa*, tendría que poner:

```
db.nombreColeccion.drop()
```

4.2 Sentencias CRUD en MongoDB

Las sentencias *CRUD (**C**reate, **R**ead, **U**pdate y **D**elete)* son las que nos permiten hacer las operaciones básicas de *consulta*, *inserción*, *modificación* y *borrado* de datos. Serían por tanto el equivalente a las sentencias *SELECT, INSERT, UPDATE y DELETE* de *SQL*.

4.2.1 Búsqueda de documentos

4.2.1.1 El método .find()

El método ***.find()*** es usado para buscar *documentos* en una *colección* y es el equivalente al *SELECT* de *SQL*. El siguiente comando mostrará todos los *documentos* de la *colección* llamada *posts* dentro de la base de datos que tengamos activa en ese momento.

```
db.posts.find()
```

Filtrar en find()

Si queremos filtrar los *documentos* por algún criterio, podemos añadirle en el primer parámetro, un *objeto JSON* que los especifique. Por ejemplo:

```
db.guitarists.find({ last_name: "Santana" })
```

Si quiero filtrar utilizando los operadores: ***mayor ($gt)***, ***menor ($lt)***, ***mayor o igual ($gte)*** o ***menor o igual ($lte)***, procedería de la siguiente forma:

```
db.inventory.find( { quantity: { $gte: 20 } } )
```

Orden de documentos

La ordenación de los *documentos* resultantes puede hacerse con el ***método sort().*** Éste puede ser añadido a una consulta para ordenar los *documentos* resultantes por el criterio que le indiquemos. La ordenación puede hacerse por uno o más campos y puede establecerse en orden ascendente (***valor 1***) o descendente (***valor -1***). Por ejemplo:

```
db.documentaries.find().sort({ release_year: -1 })
```

Restringir campos visibles

Cuando usamos el *método find()*, podemos limitar los campos del *documento* que queremos devolver. Para ello, en un segundo parámetro, podemos indicarle que campos queremos excluir (con el ***valor 0***) o incluir (con el ***valor 1***). Por ejemplo:

```
db.restaurants.find( {}, { origin: 0, original_cost: 0 } )
```

La sentencia anterior mostraría todos los campos excepto los llamados *'origin'* y *'original_cost'*.

Si lo que quiero es mostrar sólo los campos '*origin*' y '*original_cost*' pondría:

```
db.restaurants.find( {}, { origin: 1, original_cost: 1 } )
```

No es posible combinar en una misma expresión valores inclusivos y exclusivos. La única excepción a la regla anterior es si referenciamos al campo *'_id'*.

```
db.artists.find( { }, { last_name: 1, movement: 1, _id: 0 } )
```

Documento embebido

Un *documento embebido* es un *documento* que forma parte de otro de nivel superior. Consideremos cómo se define el valor del campo '*teacher*' en el siguiente ejemplo:

```
{
  course: "CS101",
  course_number: 10401,
  teacher: { first_name: "Sam", last_name: "Juarez" },
  branch: "Computer Science"
}
```

Es posible referenciar un contenido embebido dentro de ***find***() utilizando el operador ***'.'*** . Por ejemplo:

```
db.employees.find( { "teacher.last_name": "Juarez" } )
```

Múltiples condiciones sobre valor

Existe la opción de poder aplicar varias condiciones sobre un valor determinado, de manera que deban cumplirse todas para que el documento pase el filtro. El siguiente ejemplo simula el operador *BETWEEN* de *SQL*.

```
db.ufc_contestants.find({  championship_years: { $gte: 2004, $lte: 2008 } })
```

$and
$or
$in

Si quiero *poner* un filtro compuesto por la combinación de varios campos puedo utilizar los operadores ***$and***, ***$or*** y ***$in***. A continuación, se muestran algunos ejemplos de cómo utilizarlos:

```
db.colName.find({Campo1:'V1',Campo2:'V2'})    // Es equivalente a la siguiente
db. colName.find({ $and:[{Campo1:'V1',Campo2:'V2'}] })

db. colName.find({ $or:[{ Campo1:'V1',Campo2:'V2'}] })
db. colName.find({  Campo1: {$in:["V1", "V2"] } })
```

Arrays

Si tengo un campo cuyo valor es un array de valores. Puedo chequear dentro de ***find()*** si sus valores coinciden exactamente con los que le paso de la siguiente forma:

```
db.interviewees.find({  languages: ["C#", "Javascript", "Ruby"] })
```

Arrays
$all

El operador ***$all*** selecciona aquellos documentos cuyo valor para un campo de tipo array, contiene al menos todos los elementos especificados en el array pasado, sin importar en qué orden. Por ejemplo:

```
db.interviewees.find({  languages: {$all: ["C#", "Javascript", "Ruby"]} })
```

Arrays
$elemMatch

El operador ***$elemMatch*** permite filtrar *documentos* que para un campo de tipo array, alguno de sus datos cumpla las condiciones que le indiquemos. Por ejemplo:

```
db.music_fest.find({  bands: { $elemMatch: { active: true, formation_year: { $gte: 1990}  }} })
db.exams.find( { results: { $elemMatch:{$in:[80,90]}}})
```

Si referenciamos el *$elemMatch* en la parte de los campos a devolver, en lugar de devolver todo el documento, devolvería únicamente la primera ocurrencia del array.

```
db.exams.find( {asignatura:'Maths'} , { results: { $elemMatch:{$in:[80,90]}}})
```

Si quisiera que me devolviera todas las ocurrencias del array, habría que hacerlo con la operación *aggregate*, haciendo uso del *$unwind* y filtrando luego con *$match*.

4.2.2 Inserción de documentos

4.2.2.1 El campo _id

El campo ***_id*** es asignado automáticamente a cada *documento* como un identificador único. **No puede haber dentro de una *colección* dos *documentos* con el mismo *_id*.**

Lo normal, es que cuando insertamos un *documento*, no especifiquemos ningún atributo *_id*, aunque podríamos hacerlo. Es el propio *MongoDB* quien se encarga de generar uno automáticamente y añadírselo al *documento*.

4.2.2.2 El método .insertOne()

Este método inserta un *documento* en una *colección*. Necesita un único parámetro en el que le pasemos el *documento* que queremos insertar.

```
db.employees.insertOne({ name: "Josh", surname: "Brown" })
```

OJO. Si la *colección* donde queremos insertar el *documento* no existiera la crearía.

4.2.2.3 El método .insertMany()

Permite insertar varios *documentos* en una *colección*. Necesita que le pasemos un array con los *documentos* a insertar.

```
db.pets.insertMany([
 { name: "Migo", type: "Dog" },
 { name: "Snowball", type: "Cat" }
])
```

4.2.3 Modificación de documentos

4.2.3.1 El método .updateOne()

Actualiza un único *documento* que se ajuste al filtro proporcionado. Requiere dos parámetros: el filtro para localizarlo y la actualización a realizar. En el caso de que haya varios *documentos* que se ajusten al filtro indicado, actualizará el primero de ellos. Por ejemplo:

```
db.pets.updateOne(
 { name: "Snowball"},
 { $set: { type: "Bengal Cat"}}
)
```

Operador $set

El *operador* ***$set***, permite modificar el valor de uno o varios campos del documento localizado. Si el campo que le indico que quiero modificar no existe, lo crearía. Ejemplos:

db.estaciones.updateOne({Municipio:'Totana'},{$set: {Fax:'123456'} })
db.estaciones.updateOne({Municipio:'Totana'},{$set: {Fax:'123457',Email:'aaa@aa.es'} })
db.estaciones.updateOne({Municipio:'Totana'},{$set: {NuevoCampo:'x'} })

Operador $unset

Si lo que quiero es borrar un campo del documento, debería usar el operador ***$unset.*** Por ejemplo:

db.estaciones.updateOne({Municipio:'Totana'},{$unset: {CampoBorrar:1} })

Campos de tipo array

Si tuviéramos que modificar el valor de un elemento de un array para un campo, podemos utilizar el carácter ***'.'*** y el número de elemento. Por ejemplo, si tuviéramos el *documento*:

```
{
 name: "Superman"
 powers: ["Flight", "Lasers", "Super Strength", "Healing"]
}
```

Podríamos escribir la sentencia de actualización (el primer elemento es el 0 y hay que poner "):
db.superheros.updateOne({ name: "Superman"}, { $set: {"powers.1": "Laser Eyes"} })

Arrays -$push añadir

El operador ***$push*** puede usarse en ***.updateOne()*** y ***.updateMany()*** para añadir un valor a un campo de tipo array.

db.students.updateOne({ _id: 1 }, { $push: { scores: 89 }})

Por defecto, lo añade al final del array. Si quisiera añadírselo en una posición concreta utilizaría:

db.estaciones.updateOne({Municipio:'Totana'},{$push: {Fax:{$each:888888,$position:0}} })

Donde ***$each*** especifica el valor o valores ([v1,v2]) que quiero insertar y ***$position*** indica dónde hacerlo

Arrays -$push $pop-quitar

También tengo mecanismos para eliminar ciertos datos de un campo de tipo array. Por ejemplo:

- Si quiero quitar el primero:
  db.estaciones.updateOne({Municipio:'Totana'},{$pop: {Fax:-1}})

- Si quiero quitar el último:
  db.estaciones.updateOne({Municipio:'Totana'},{$pop: {Fax:1}})

- Si quiero quitar un valor concreto:
  db.estaciones.updateOne({Municipio:'Totana'},{$pull: {Fax:9999999}})

Parámetro upsert

El parámetro *upsert* es opcional. Permite combinar la operación *update* e *insert*, de manera que si no se localiza ningún *documento* con el criterio indicado para hacer la actualización, se insertaría el nuevo *documento {name:"Eric Wikstrom", department: "Software Engineering"}* .

```
db.employee.updateOne(
 { name:"Eric Wikstrom" },
 { $set: { department: "Software Engineering" }},
 { upsert: true }
)
```

4.2.3.2 El método .updateMany()

Funciona igual que el método ***.updateOne()*** con la diferencia de que en este caso actualizaría todos los *documentos* que concuerden con el criterio expresado.

```
db.employees.updateMany(
 { salary: 70000 },
 { $set: { salary: 85000 }}
)
```

4.2.3.3 El método .replaceOne()

Este método permite sustituir completamente el primer *documento* que se ajuste al filtro pasado por el *documento* que se incluye en el segundo parámetro. El campo ***_id*** lo mantendría. Por ejemplo:

```
db.inventory.replaceOne(
 { name: "Chango Chile"},
 { name: "Chango Chili", scoville: 9000 }
)
```

4.3 Borrado de documentos

4.3.1.1 El método .deleteOne()

Permite borrar el primer *documento* de una *colección* que se ajuste al filtro indicado. Por ejemplo:

```
db.recipes.deleteOne({   title: "King Burger Extravaganza"  });
```

4.3.1.2 El método .deleteMany()

Permite borrar todos los *documentos* de una *colección* que se ajusten al filtro indicado. Por ejemplo:

```
db.televisions.deleteMany({   status: "sold" })
```

5 Operaciones de agregación. Método .aggregate()

Las operaciones de agregación procesan grupos de *documentos* y generan *documentos* resumen para cada uno de los grupos.

Las operaciones de agregación se van realizando en una serie de etapas, a través de lo que se conoce como un ***pipeline (tubería)***, donde cada etapa trabaja sobre unos *documentos* de entrada y genera otros *documentos* de salida que sirven de entrada a la siguiente etapa.

El método encargado de hacer todo esto es ***.aggregate()***. Internamente, permite definir una gran variedad de tipos de etapas diferentes. En el siguiente enlace podemos ver una descripción de cada una de ellas:
https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/#std-label-aggregation-pipeline-operator-reference

Las etapas más usadas en las operaciones de agregación son:

- ***$group***. Permite hacer grupos de los *documentos* de entrada por el criterio que le indiquemos. Sobre cada grupo se aplica una o varias *funciones acumulador* para resumirlo, obteniendo un *documento* representativo de cada grupo. Es el equivalente de la fase de ***GROUP BY*** y de la aplicación de ***funciones resumen*** en el ***SELECT*** de ***SQL***.
- ***$match***. Se utiliza para filtrar *documentos*. Es el equivalente al ***WHERE*** y al ***HAVING*** de SQL, en base al orden en que se aplique en el *pipeline*.
- ***$sort***. Reordena los *documentos* por el criterio que le indiquemos.
- ***$lookup***. Permite enlazar con *documentos* de otra *colección.*
- ***$addfields***. Permite añadir nuevos campos al documento. El documento de salida contendría todos los campos de los documentos de entrada más los nuevos.
- ***$project***. Permite definir qué campos de los documentos de entrada, van a estar en el documento de salida.
- ***$limit***. En lugar de quedarse con todos los *documentos*, nos quedaríamos únicamente con un número de ellos. Equivalente al ***LIMIT*** de *SQL*

Las funciones más frecuentes que se les aplican a los distintos grupos para resumirlos son:

- ***$count***. Cuenta el número de *documentos* en un grupo
- ***$max***. Retorna el máximo valor de un grupo
- ***$maxN***. Retorna los N máximos valores de un grupo
- ***$min***. Retorna el mínimo valor de un grupo
- ***$minN***. Retorna los N mínimos valores de un grupo
- ***$sum***. Retorna la suma de un valor determinado
- ***$avg***. Retorna la media aritmética para un valor determinado

Por ejemplo, si quisiera representar en *MongoDB* el equivalente a las siguientes sentencias *SQL* escribiría lo siguiente:

SELECT fabricante, SUM(precio) AS totalPrecio *FROM productos* *GROUP BY fabricante*	*db.productos.aggregate([* *{ // Etapa 1* *$group:* *{* *_id: { "fabricante":"$fabricante" } ,* *totalPrecio: {$sum:"$precio"}* *}* *}* *])*

SELECT fabricante, COUNT(*) AS numProductos
FROM productos
WHERE precio > 1000
GROUP BY fabricante
	db.productos.aggregate([
	{ // Etapa 1
	$match: { "precio": { $gt: 1000 } }
	} ,
	{ // Etapa 2
	$group:
	{
	_id: { "fabricante":"$fabricante" } ,
	numProductos: {$count:{}}
	}
	}
	])

SELECT fabricante, COUNT(*) AS numProductos
FROM productos
WHERE precio > 1000
GROUP BY fabricante
HAVING numProductos > 10
ORDER BY numProductos DESC
	db.productos.aggregate([
	{ // Etapa 1
	$match: { "precio": { $gt: 1000 } }
	} ,
	{ // Etapa 2
	$group:
	{
	_id: { "fabricante":"$fabricante" } ,
	numProductos: {$count:{}}
	}
	},
	{ // Etapa 3
	$match: { "numProductos": { $gt: 10 } }
	} ,
	{ // Etapa 4
	$sort : { numProductos: -1 }
	}
	])

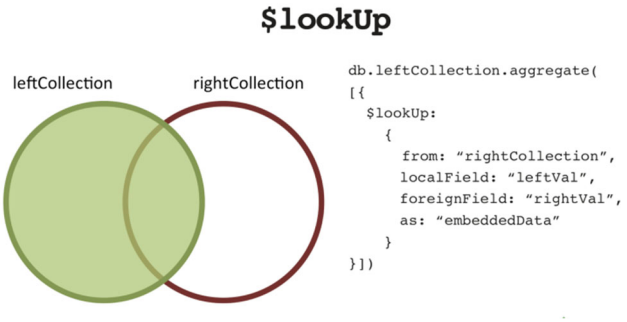
SELECT Municipio,Pedanía,COUNT(*) AS numEstaciones *FROM Estaciones* *GROUP BY Municipio,Pedanía* *HAVING numEstaciones>2*	*db.estaciones.aggregate([* *{ // Etapa 1* *$group :* *{* *_id:{municipio:"$Municipio",pedanía:"$Pedanía"},* *numEstaciones: { $count: { } }* *}* *}, // Etapa2* *{* *$match: { "numEstaciones": { $gt: 2 } }* *}* *])*

5.1 Cómo realizar operaciones de Join en MongoDB

Oficialmente, no hay *joins* en *MongoDB*, sin embargo, hay métodos para implementarlos.

El operador ***$lookup*** en las operaciones de *agregación* es el elemento a través del cual lo haremos. Este operador tiene 4 campos principales:

- ***from***. Permite indicar el nombre de la *colección* con la que queremos hacer el *join*.
- ***localField***. Campo de la *colección* que invoca el ***$lookup*** usado para el *join*
- ***foreignField***. Campo de la *colección* indicada en el ***from*** usado para el *join*.
- ***as***. Nombre del campo que se añadirá a la *colección* de entrada en la que se almacenará el resultado de los *documentos* enlazados de la otra *colección*.



En realidad, el operador ***$lookup*** lo que realiza es un ***LEFT OUTER JOIN***, ya que los *documentos* de la primera *colección* que no se vinculen con ningún *documento* de la derecha también aparecen en el resultado, pero con el campo que hemos puesto en la cláusula ***'as'*** vacío.

Si lo que queremos realmente es hacer un ***INNER JOIN***, tendremos que añadirle una nueva fase a la operación de *agregación*. En este caso, usaremos el operador ***$match***, a través del cual, podremos eliminar los *documentos* de entrada que no cumplan el criterio que le indicamos, que será que el campo añadido en el ***'as'*** del ***$lookup*** anterior, no tenga asociado ningún *documento*.

```
{ $match : { "nombreCampoAS" : { $ne : [] } } }
```

Por ejemplo, imaginemos que queremos enlazar mediante un **INNER JOIN**, los *documentos* de las *colecciones* ***Coleccion1*** y ***Colección2***, a través de un campo que tienen ambas llamado ***ID***. Escribiríamos la siguiente operación de agregación:

```
db.Coleccion1.aggregate([{
  $lookup: { from: "Coleccion2", localField: "ID",  foreignField: "ID",  as: "resultado" }
},{
 $match: {  resultado: { $ne: [] } }
}
}])
```